Exp. No: 7

GENERATIVE MODELS WITH GANS: CREATING AND TRAINING A GENERATIVE ADVERSARIAL NETWORK

Code:

```
# 1. Import Libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist
import os

tf.keras.utils.disable_interactive_logging()


# 2. Load and Preprocess the MNIST Dataset
(x_train, _), (_, _) = mnist.load_data()

x_train = x_train.reshape(
  x_train.shape[0],
  28,
  28,
  1
).astype("float32")

x_train = (x_train - 127.5) / 127.5

print("Normalized training data shape:", x_train.shape)
```

```
print(
  "Example of a normalized pixel value:",
  x_train[0, 0, 0, 0]
)



# 3. Define GAN Parameters
latent_dim = 100
batch_size = 256
EPOCHS = 200
num_examples_to_generate = 16



# 4. Build the Generator

def build_generator():
  model = keras.Sequential(name="generator")

  model.add(
    layers.Dense(
      7 * 7 * 256,
      use_bias=False,
      input_shape=(latent_dim,)
    )
  )

  model.add(layers.BatchNormalization())
  model.add(layers.LeakyReLU())

  model.add(
    layers.Reshape(
      (7, 7, 256)
    )
  )
```

```
model.add(
   layers.Conv2DTranspose(
     128,
     (5, 5),
     strides=(1, 1),
     padding="same",
     use_bias=False
   )
  )

model.add(layers.BatchNormalization())
  model.add(layers.LeakyReLU())

  model.add(
   layers.Conv2DTranspose(
     64,
     (5, 5),
     strides=(2, 2),
     padding="same",
     use_bias=False
   )
  )

  model.add(layers.BatchNormalization())
  model.add(layers.LeakyReLU())

  model.add(
   layers.Conv2DTranspose(
     1,
     (5, 5),
     strides=(2, 2),
     padding="same",
     use_bias=False,
     activation="tanh"
   )
  )
```

```
        return model

generator = build_generator()

print("\nGenerator Model Summary:")
generator.summary()


# 5. Build the Discriminator

def build_discriminator():
 model = keras.Sequential(name="discriminator")

 model.add(
  layers.Conv2D(
   64,
   (5, 5),
   strides=(2, 2),
   padding="same",
   input_shape=(28, 28, 1)
  )
 )

 model.add(layers.LeakyReLU())
 model.add(layers.Dropout(0.3))

 model.add(
  layers.Conv2D(
   128,
   (5, 5),
   strides=(2, 2),
   padding="same"
  )
 )
```

```
model.add(layers.LeakyReLU())
  model.add(layers.Dropout(0.3))

  model.add(layers.Flatten())

  model.add(
    layers.Dense(
      1,
      activation="sigmoid"
    )
  )

  return model


discriminator = build_discriminator()

print("\nDiscriminator Model Summary:")
discriminator.summary()


# 6. Define Loss Functions

cross_entropy = keras.losses.BinaryCrossentropy(
  from_logits=False
)


def discriminator_loss(real_output, fake_output):
  real_loss = cross_entropy(
    tf.ones_like(real_output),
    real_output
  )
```

```
fake_loss = cross_entropy(
   tf.zeros_like(fake_output),
   fake_output
  )

  return real_loss + fake_loss

def generator_loss(fake_output):
  return cross_entropy(
   tf.ones_like(fake_output),
   fake_output
  )



# 7. Define Optimizers

generator_optimizer = tf.keras.optimizers.Adam(
  learning_rate=1e-4
)

discriminator_optimizer = tf.keras.optimizers.Adam(
  learning_rate=1e-4
)



# 8. Define Training Step

@tf.function
def train_step(images):
  noise = tf.random.normal(
   [batch_size, latent_dim]
  )

  with tf.GradientTape() as gen_tape, \
    tf.GradientTape() as disc_tape:
```

```
generated_images = generator(
    noise,
    training=True
  )

  real_output = discriminator(
    images,
    training=True
  )

  fake_output = discriminator(
    generated_images,
    training=True
  )

  gen_loss = generator_loss(
    fake_output
  )

  disc_loss = discriminator_loss(
    real_output,
    fake_output
  )

 gradients_of_generator = gen_tape.gradient(
  gen_loss,
  generator.trainable_variables
 )

 gradients_of_discriminator = disc_tape.gradient(
  disc_loss,
  discriminator.trainable_variables
 )
```

```
generator_optimizer.apply_gradients(
   zip(
     gradients_of_generator,
     generator.trainable_variables
   )
  )

  discriminator_optimizer.apply_gradients(
   zip(
     gradients_of_discriminator,
     discriminator.trainable_variables
   )
  )

  return gen_loss, disc_loss


# 9. Generate and Save Images

def generate_and_save_images(
  model,
  epoch,
  test_input
):
  predictions = model(
    test_input,
    training=False
  )
   predictions_rescaled = (
    predictions * 0.5
  ) + 0.5

  fig = plt.figure(
    figsize=(4, 4)
  )
```

```
for i in range(
    predictions.shape[0]
  ):
    plt.subplot(
      4,
      4,
      i + 1
    )

    plt.imshow(
      predictions_rescaled[
        i, :, :, 0
      ],
      cmap="gray"
    )

    plt.axis("off")

  plt.suptitle(
    f"Epoch {epoch}",
    fontsize=16
  )

  if not os.path.exists("images"):
    os.makedirs("images")

  plt.savefig(
    f"images/image_at_epoch_{epoch:04d}.png"
  )

  plt.show()
  plt.close()
```

10. Prepare Training Dataset

```
train_dataset = (
  tf.data.Dataset
  .from_tensor_slices(x_train)
  .shuffle(x_train.shape[0])
  .batch(batch_size)
)


# 11. Create Seed Images

seed = tf.random.normal(
  [
    num_examples_to_generate,
    latent_dim
  ]
)


# 12. Train the GAN

def train(dataset, epochs):
  print("\nBeginning GAN Training...")

  for epoch in range(epochs):
    gen_loss_list = []
    disc_loss_list = []

    for image_batch in dataset:
      gen_loss, disc_loss = train_step(
        image_batch
      )

      gen_loss_list.append(
        gen_loss.numpy()
      )
```

```
    avg_gen_loss = np.mean(
      gen_loss_list
    )

    avg_disc_loss = np.mean(
      disc_loss_list
    )

    print(
      f"Epoch {epoch + 1}/{epochs} - "
      f"Generator Loss: {avg_gen_loss:.4f} - "
      f"Discriminator Loss: {avg_disc_loss:.4f}"
    )

    if (epoch + 1) % 20 == 0:
      generate_and_save_images(
        generator,
        epoch + 1,
        seed
      )

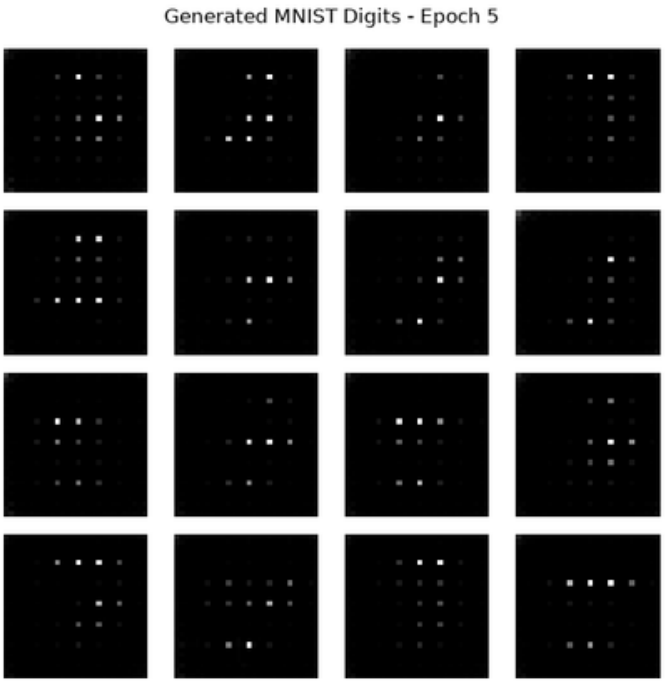
  print(
    "\nTraining complete. "
    "Generating final images."
  )

  generate_and_save_images(
    generator,
    epochs,
    seed
  )

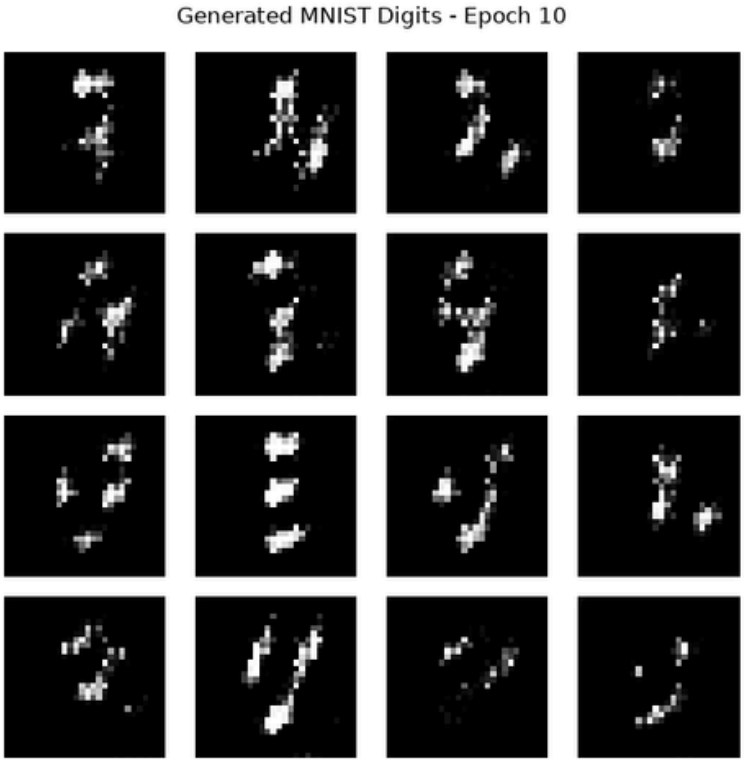
# 13. Run GAN Training
train(
  train_dataset,
  EPOCHS
```

Output:

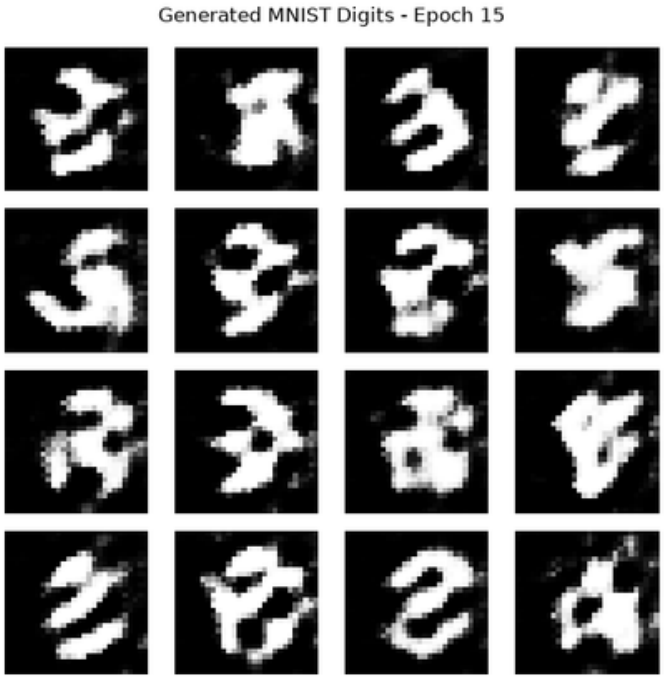
Beginning GAN Training...
Epoch 1/20 - Generator Loss: 0.6060 - Discriminator Loss: 1.1249
Epoch 2/20 - Generator Loss: 0.8913 - Discriminator Loss: 0.9839
Epoch 3/20 - Generator Loss: 0.8228 - Discriminator Loss: 1.1593
Epoch 4/20 - Generator Loss: 0.8924 - Discriminator Loss: 1.1915
Epoch 5/20 - Generator Loss: 0.8146 - Discriminator Loss: 1.2881



Saved: images/image_at_epoch_0005.png
Epoch 6/20 - Generator Loss: 1.1308 - Discriminator Loss: 0.9871
Epoch 7/20 - Generator Loss: 0.8734 - Discriminator Loss: 1.3042
Epoch 8/20 - Generator Loss: 0.8371 - Discriminator Loss: 1.2605
Epoch 9/20 - Generator Loss: 0.8321 - Discriminator Loss: 1.2430
Epoch 10/20 - Generator Loss: 0.8421 - Discriminator Loss: 1.2410



Saved: images/image_at_epoch_0010.png
Epoch 11/20 - Generator Loss: 0.8421 - Discriminator Loss: 1.2630
Epoch 12/20 - Generator Loss: 0.8251 - Discriminator Loss: 1.2984
Epoch 13/20 - Generator Loss: 0.8029 - Discriminator Loss: 1.3403
Epoch 14/20 - Generator Loss: 0.7865 - Discriminator Loss: 1.3261
Epoch 15/20 - Generator Loss: 0.8336 - Discriminator Loss: 1.3532



Saved: images/image_at_epoch_0015.png
Epoch 16/20 - Generator Loss: 0.7886 - Discriminator Loss: 1.2898
Epoch 17/20 - Generator Loss: 0.8382 - Discriminator Loss: 1.2489
Epoch 18/20 - Generator Loss: 0.8970 - Discriminator Loss: 1.2265
Epoch 19/20 - Generator Loss: 0.8270 - Discriminator Loss: 1.2875
Epoch 20/20 - Generator Loss: 0.8469 - Discriminator Loss: 1.3086

